

Diagnosing Spark Join and Aggregation Results

(a) Why North is 60 instead of the intended North total

The customer dataset violates its intended grain of one row per customer: customer_id 1 appears twice, and both rows are labelled North. A standard join therefore matches each event for customer 1 to both customer rows. The event values 10 and 20 are each duplicated in the joined result, so the North aggregation becomes $10 + 20 + 10 + 20 = 60$, rather than $10 + 20 = 30$. Customer 2 has one matching row, so South remains 5.

(b) Defensible correction and validation

Before joining, enforce the dimension table's intended key: retain one customer row per customer_id only after checking that duplicated records are truly equivalent. Here, the duplicate rows for customer_id 1 are both North, so one can be removed safely. If duplicate rows carried conflicting regions, the data should be investigated and resolved rather than arbitrarily deduplicated. A validation check should require customer_id to be unique in the customer dataset (for example, compare `count(*)` with `countDistinct(customer_id)`, or flag any customer_id with `count > 1`) before the join. This prevents one-to-many joins from silently multiplying event values.

(c) Regional totals after correction

Region	Calculation	Correct total
North	$10 + 20$	30
South	5	5

Correct totals: North = 30 and South = 5.